Stats-506 Problem Set #3

GitHub Repository: https://github.com/zkl2002/Stats-506/

Zhekai Liu

```
library(haven)
library(dplyr)
library(knitr)
library(DBI)
library(RSQLite)
library(microbenchmark)
library(pscl)
```

Problem 1

(a).

```
# read files
aux  <- read_xpt("AUX_I.XPT")
demo <- read_xpt("DEMO_I.XPT")
# merge data set
df <- inner_join(aux, demo, by = "SEQN")
dim(df)
```

```
[1] 4582  119
```

The merged date frame has 4582 rows and 119 columns.

(b).

```r
# switch missing code to NA
df$DMDCITZN[df$DMDCITZN %in% c(7, 9)] <- NA_integer_
df$INDHHIN2[df$INDHHIN2 %in% c(77, 99)] <- NA_integer_

# gender as factor
df$gender <- factor(df$RIAGENDR, levels = c(1, 2),
                        labels = c("Male", "Female"))
# citizenship as factor
df$citizen <- factor(df$DMDCITZN, levels = c(1, 2),
                        labels = c("Citizen", "Not citizen"))
# children under 5 years old
df$children_under5 <- factor(df$DMDHHSZA,
                             levels = c(0, 1, 2, 3),
                             labels = c("0", "1", "2", "3 or more"),
                             ordered = TRUE)

# household income
codes  <- c(1:10, 12, 13, 14, 15)
labels <- c("$0 to $4,999",
            "$5,000 to $9,999",
            "$10,000 to $14,999",
            "$15,000 to $19,999",
            "$20,000 to $24,999",
            "$25,000 to $34,999",
            "$35,000 to $44,999",
            "$45,000 to $54,999",
            "$55,000 to $64,999",
            "$65,000 to $74,999",
            "$20,000 and Over",
            "Under $20,000",
            "$75,000 to $99,999",
            "$100,000 and Over")

df$household_income <- factor(df$INDHHIN2, levels = codes,
                              labels = labels, ordered = TRUE)

df_clean <- df[c("SEQN", "gender", "citizen",
                 "children_under5", "household_income")]
```

`df_clean` is the clean up data frame and all categorical values converted to factor with informative values.

(c).

```
# right ear
xR <- df$AUXTWIDR
xR[xR %in% c(555, 777, 888)] <- NA_integer_
xR[xR < 0 | xR > 500] <- NA_integer_
df_clean$tw_right <- as.integer(xR)

# left ear
xL <- df$AUXTWIDL
xL[xL %in% c(555, 777, 888)] <- NA_integer_
xL[xL < 0 | xL > 500] <- NA_integer_
df_clean$tw_left <- as.integer(xL)

# children number as continuous
df_clean$children <- df$DMDHHSZA
df_clean$children <- as.integer(df_clean$children)

# income as continuous
inc <- df$INDHHIN2
inc[inc %in% c(77, 99)] <- NA_integer_

codes_keep <- c(1:10, 12:15)
mids <- c( 2500, 7500, 12500, 17500, 22500, 30000,
           40000, 50000, 60000, 70000, 20000, 10000,
           87500, 100000)

df_clean$income <- NA_real_
idx <- match(inc, codes_keep)
df_clean$income[!is.na(idx)] <- mids[idx[!is.na(idx)]]
```

We treat `children` as a continuous count by coding "3 or more" as 3, and we convert `household income` to a continuous measure using category midpoints (codes 77/99 set to missing). If some household income is over 20,000 or over 100,000, we just set as 20,000 and 100,000.

```
# models
m1R <- glm(tw_right ~ gender, data = df_clean,
           family = poisson, na.action = na.omit)

m2R <- glm(tw_right ~ gender + citizen + children + income,
           data = df_clean, family = poisson, na.action = na.omit)
```

```r
m1L <- glm(tw_left  ~ gender, data = df_clean,
           family = poisson, na.action = na.omit)

m2L <- glm(tw_left  ~ gender + citizen + children + income,
           data = df_clean, family = poisson, na.action = na.omit)
```

```r
b1R <- exp(coef(m1R))
b2R <- exp(coef(m2R))
b1L <- exp(coef(m1L))
b2L <- exp(coef(m2L))

ref_len <- length(b2R)
coeftable <- cbind(
  c(b1R, rep(NA, ref_len - length(b1R))),
  c(b2R, rep(NA, ref_len - length(b2R))),
  c(b1L, rep(NA, ref_len - length(b1L))),
  c(b2L, rep(NA, ref_len - length(b2L)))
)
rownames(coeftable) <- names(b2R)
colnames(coeftable) <- c("1R","2R","1L","2L")

coeftable <- round(coeftable, 3)
mode(coeftable) <- "character"; coeftable[is.na(coeftable)] <- ""
kable(coeftable, align = "cccc", caption = "IRR (exp(beta)) including baseline")
```

Table 1: IRR (exp(beta)) including baseline

	1R	2R	1L	2L
(Intercept)	84.317	87.395	84.758	87.31
genderFemale	1.009	1.013	1.013	1.015
citizenNot citizen		1.071		1.041
children		1		0.982
income		1		1

```r
N <- c(nobs(m1R), nobs(m2R), nobs(m1L), nobs(m2L))

pseudo <- c(pR2(m1R)["McFadden"],
            pR2(m2R)["McFadden"],
            pR2(m1L)["McFadden"],
            pR2(m2L)["McFadden"])
```

```
fitting null model for pseudo-r2
fitting null model for pseudo-r2
fitting null model for pseudo-r2
fitting null model for pseudo-r2
```

```
AICv <- c(AIC(m1R), AIC(m2R), AIC(m1L), AIC(m2L))

stats_tab <- rbind(
  "Sample size" = sprintf("%d",   N),
  "Pseudo R^2"  = sprintf("%.5f", pseudo),
  "AIC"         = sprintf("%.2f", AICv)
)

colnames(stats_tab) <- c("1R","2R","1L","2L")

knitr::kable(stats_tab, align = "lrrrr",
             caption = "Models sample size, pseudo-R², and AIC")
```

Table 2: Models sample size, pseudo-R^2, and AIC

	1R	2R	1L	2L
Sample size	4149	3886	4103	3847
Pseudo R^2	0.00008	0.00718	0.00015	0.00323
AIC	96618.48	90028.77	98685.15	91798.98

(d).

```
summary(m2L)
```

```
Call:
glm(formula = tw_left ~ gender + citizen + children + income,
    family = poisson, data = df_clean, na.action = na.omit)

Coefficients:
                     Estimate Std. Error  z value Pr(>|z|)
(Intercept)         4.469e+00  4.158e-03 1074.903  < 2e-16 ***
genderFemale        1.499e-02  3.510e-03    4.270 1.95e-05 ***
citizenNot citizen  4.028e-02  4.478e-03    8.996  < 2e-16 ***
```

```
children           -1.796e-02  2.644e-03   -6.791 1.11e-11 ***
income             -6.391e-07  5.467e-08  -11.691  < 2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for poisson family taken to be 1)

    Null deviance: 68282  on 3846  degrees of freedom
Residual deviance: 67985  on 3842  degrees of freedom
  (   735    )
AIC: 91799

Number of Fisher Scoring iterations: 4
```

The result is statistically significant, and estimate coefficient for females vs. males is **greater than 0**. This provides evidence that, holding citizenship, number of young children, and household income constant, females have a **higher expected tympanometric width in the left ear** than males.

```r
keep <- complete.cases(df_clean[, c("tw_left","gender",
                                    "citizen","children","income")])
dd   <- df_clean[keep, ]

new_m <- data.frame(
  gender   = factor("Male",   levels=levels(dd$gender)),
  citizen  = factor("Citizen",levels=levels(dd$citizen)),
  children = mean(dd$children),
  income   = mean(dd$income, na.rm=TRUE)
)
new_f <- new_m
new_f$gender <- factor("Female", levels=levels(dd$gender))

mu_m <- exp(predict(m2L, new_m, type = "link"))
mu_f <- exp(predict(m2L, new_f, type = "link"))
ratio <- mu_f / mu_m

p2 <- summary(m2L)$coef["genderFemale","Pr(>|z|)"]

data.frame(
  Setting = "Citizen; children=mean; income=mean",
  Pred_Male   = round(mu_m, 2),
  Pred_Female = round(mu_f, 2),
```

```
  Ratio_F_over_M = round(ratio, 3),
  p_value = signif(p2, 3)
)
```

```
                              Setting Pred_Male Pred_Female Ratio_F_over_M
1 Citizen; children=mean; income=mean     83.91       85.18          1.015
   p_value
1 1.95e-05
```

By controlling for citizenship, number of children, and household income, females have a higher expected tympanometric width than males. The estimated **incidence rate ratio** for females vs. males is **1.015** (**1.5%** higher), with p = $\mathbf{1.95 \times 10^{-5}}$, indicating a statistically significant difference.

Problem 2 - Sakila

(a).

```
sakila <- dbConnect(RSQLite::SQLite(), "sakila_master.db")
```

R code method:

```
# Output of problem a with R code
R_a <- function() {
  cust <- dbGetQuery(sakila, "SELECT customer_id,
                      store_id, active FROM customer;")
  tot <- aggregate(customer_id ~ store_id, cust, length)
  names(tot)[2] <- "n_customers"

  act <- aggregate(as.numeric(active) * 100 ~ store_id, cust, mean)
  names(act)[2] <- "active_pct"

  out <- merge(tot, act, by = "store_id", sort = TRUE)
  out
}
R_a()
```

```
  store_id n_customers active_pct
1        1         326   97.54601
2        2         273   97.43590
```

SQL method:

```
SQL_a <- function() {
  dbGetQuery(sakila, "
    SELECT store_id,
           COUNT(customer_id) AS n_customers,
           ROUND(AVG(active) * 100.0, 4) AS active_pct
    FROM customer
    GROUP BY store_id
    ORDER BY store_id;")
}
SQL_a()
```

```
  store_id n_customers active_pct
1        1         326    97.5460
2        2         273    97.4359
```

```
bench_a <- microbenchmark(
  R   = R_a(),
  SQL = SQL_a()
)
print(bench_a)
```

```
Unit: microseconds
 expr    min      lq     mean  median      uq     max neval
    R 4417.1 4677.65 5138.275 4825.25 5005.95 15899.2   100
  SQL  770.5  812.70 1012.189 1093.85 1114.75  2598.4   100
```

Both methods give the same result, and SQL is faster.

(b).

R method:

```
# Output of problem b with R code
R_b <- function(){
  staff <- dbGetQuery(sakila, "SELECT staff_id, first_name,
                          last_name, address_id FROM staff;")
  address <- dbGetQuery(sakila, "SELECT address_id, city_id FROM address;")
  city <- dbGetQuery(sakila, "SELECT city_id, country_id FROM city;")
```

```
  country <- dbGetQuery(sakila, "SELECT country_id, country FROM country;")
  staff_addr <- merge(staff, address, by = "address_id", all.x = TRUE)
  staff_city <- merge(staff_addr, city, by = "city_id", all.x = TRUE)
  staff_country <- merge(staff_city, country, by = "country_id", all.x = TRUE)
  res <- staff_country[, c("staff_id", "first_name", "last_name", "country")]
  res <- res[order(res$staff_id), ]
  rownames(res) <- NULL
  kable(res, align = "cccc")}
R_b()
```

staff_id	first_name	last_name	country
1	Mike	Hillyer	Canada
2	Jon	Stephens	Australia

SQL method:

```
SQL_b <- function(){
    res_sql <- dbGetQuery(sakila, "
    SELECT s.staff_id,
           s.first_name,
           s.last_name,
           co.country AS country
    FROM staff AS s
    JOIN address AS a ON a.address_id = s.address_id
    JOIN city    AS c ON c.city_id    = a.city_id
    JOIN country AS co ON co.country_id = c.country_id
    ORDER BY s.staff_id;
  ")
  kable(res_sql, align = "cccc")
}
SQL_b()
```

staff_id	first_name	last_name	country
1	Mike	Hillyer	Canada
2	Jon	Stephens	Australia

```
bench_b <- microbenchmark(
  R = R_b(),
```

```
  SQL = SQL_b()
)
print(bench_b)
```

```
Unit: milliseconds
 expr    min      lq     mean  median      uq     max neval
    R 8.2420 8.38485 9.020976 8.50070 9.07405 13.2217   100
  SQL 3.9691 4.09300 4.267890 4.13455 4.21870  6.9026   100
```

Both methods give the same result, and SQL is faster.

(c).

```
# Output of problem c with R code
R_c <- function(){
  pay <- dbGetQuery(sakila, "SELECT rental_id, amount FROM payment;")
  rent <- dbGetQuery(sakila, "SELECT rental_id, inventory_id FROM rental;")
  inv <- dbGetQuery(sakila, "SELECT inventory_id, film_id FROM inventory;")
  film <- dbGetQuery(sakila, "SELECT film_id, title FROM film;")
  sum_pay <- aggregate(amount ~ rental_id, pay, sum)
  pay_max <- max(sum_pay$amount)
  rental_max <- sum_pay$rental_id[sum_pay$amount == pay_max]
  inv_max <- unique(rent$inventory_id[rent$rental_id %in% rental_max])
  film_max <- unique(inv$film_id[inv$inventory_id %in% inv_max])
  film_names <- film$title[film$film_id %in% film_max]
  film_names
}
R_c()
```

```
[1] "FLINTSTONES HAPPINESS" "MIDSUMMER GROUNDHOG"   "MINE TITANS"
[4] "SCORPION APOLLO"       "SHOW LORD"             "STING PERSONAL"
[7] "TIES HUNGER"           "TRAP GUYS"             "VIRTUAL SPOILERS"
```

SQL method:

```
SQL_c <- function(){
  dbGetQuery(sakila, "
  SELECT DISTINCT f.title
```

```
  FROM (
    SELECT rental_id, SUM(amount) AS total_amount
    FROM payment
    GROUP BY rental_id
  ) AS t
  JOIN rental r ON r.rental_id = t.rental_id
  JOIN inventory i ON i.inventory_id = r.inventory_id
  JOIN film f ON f.film_id = i.film_id
  WHERE t.total_amount = (
    SELECT MAX(total_amount)
    FROM (
      SELECT SUM(amount) AS total_amount
      FROM payment
      GROUP BY rental_id
    )
  )
  ORDER BY f.title;
  ")
}
c(SQL_c())
```

```
$title
[1] "FLINTSTONES HAPPINESS" "MIDSUMMER GROUNDHOG"   "MINE TITANS"
[4] "SCORPION APOLLO"       "SHOW LORD"             "STING PERSONAL"
[7] "TIES HUNGER"           "TRAP GUYS"             "VIRTUAL SPOILERS"
```

```
bench_c <- microbenchmark(
  R = R_c(),
  SQL = SQL_c()
)
print(bench_c)
```

```
Unit: milliseconds
 expr      min        lq      mean    median        uq      max neval
    R 277.5869 284.50130 301.77817 288.67390 292.68175 492.1661   100
  SQL  20.9718  21.71205  24.79157  24.88175  25.53925  42.9173   100
```

Both methods give the same result, and SQL is faster.

Problem 3 - Australian Records

```
data <- read.csv("Au-500.csv")
```

(a).

```
length(data$web[grepl("com$", data$web)])/nrow(data)
```

```
[1] 0
```

No website is .com's .

(b).

```
email <- data$email
domain <- tolower(trimws(sub("^[^@]*@", "", email)))
tab <- sort(table(domain), decreasing = TRUE)
kable(head(tab))
```

domain	Freq
hotmail.com	114
gmail.com	102
yahoo.com	84
agar.net.au	1
agney.net.au	1
ahlborn.com.au	1

hotmail.com is the most common domain name amongst the email addresses.

(c).

```
company <- data$company_name
x1 <- gsub("[,[:space:]]", "", company)
nonalpha1 <- grepl("[^A-Za-z]", x1)
mean(nonalpha1)
```

```
[1] 0.09
```

```
x2 <- gsub("[,[:space:]&]", "", company)
nonalpha2 <- grepl("[^A-Za-z]", x2)
mean(nonalpha2)
```

```
[1] 0.008
```

Excluding commas and whitespace, **9%** of company names contain a non-alphabetic character.

If we also exclude ampersands ("&"), the proportion drops to **0.8%**

(d).

```
d1  <- gsub("\\D", "", data$phone1)
ok1 <- nchar(d1) == 10
data$phone1 <- ifelse(ok1,
                      sub("^(\\d{4})(\\d{3})(\\d{3})$",
                          "\\1-\\2-\\3", d1),
                      data$phone1)

d2  <- gsub("\\D", "", data$phone2)
ok2 <- nchar(d2) == 10
data$phone2 <- ifelse(ok2,
                      sub("^(\\d{4})(\\d{3})(\\d{3})$",
                          "\\1-\\2-\\3", d2),
                      data$phone2)

cat("phone1 (first 10):\n"); print(head(data$phone1, 10))
```

```
phone1 (first 10):

 [1] "0381-749-123" "0799-973-366" "0855-589-019" "0260-444-682" "0214-556-085"
 [6] "0878-681-355" "0865-228-931" "0252-269-402" "0731-849-989" "0868-904-661"
```

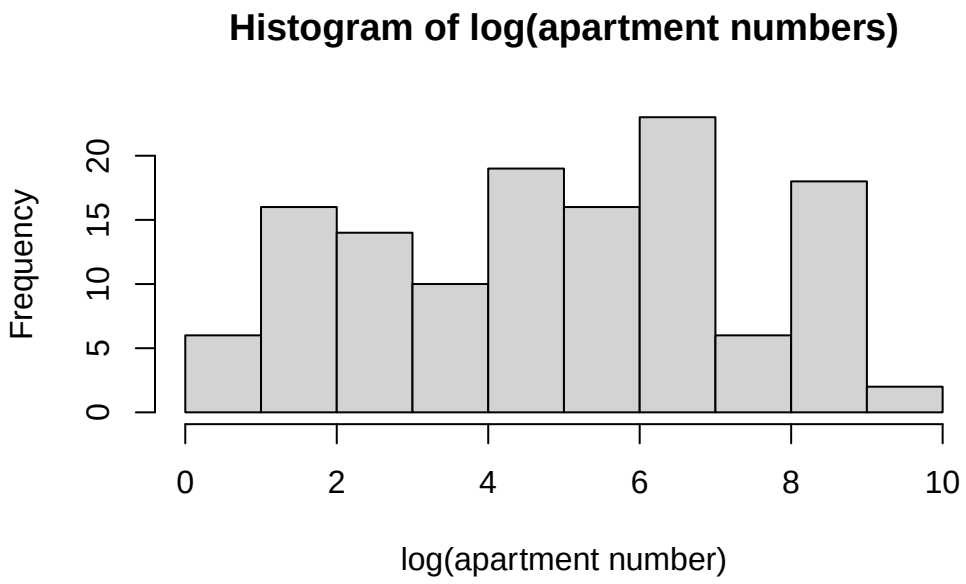
```
cat("\nphone2 (first 10):\n"); print(head(data$phone2, 10))
```

```
phone2 (first 10):

 [1] "0458-665-290" "0497-622-620" "0427-885-282" "0443-795-912" "0453-666-885"
 [6] "0451-966-921" "0427-991-688" "0415-961-606" "0411-732-965" "0461-862-457"
```

(e).

```
addr <- data$address
m <- regexpr("\\d+\\s*$", addr)
apt <- rep(NA_integer_, length(addr))
apt[m > 0] <- as.integer(regmatches(addr, m))
apt_clean <- apt[!is.na(apt) & apt > 0]
hist(log(apt_clean),
     main = "Histogram of log(apartment numbers)",
     xlab  = "log(apartment number)")
```



(f).

```
table(substr(as.character(apt_clean), 1, 1))
```

```
 1  2  3  4  5  6  7  8  9
12 19 16 13 14 18  8 11 19
```

By examining the first digit of each apartment number, its distribution aligns more closely with a uniform distribution than a decreasing one. Consequently, this data clearly violates Benford's Law, appearing more like artificially generated numbers than naturally occurring real-world numerical data.